

Source-Grounded Study Notes — Pipeline Walkthrough

A 4-step no-code research pipeline, built in a Claude Project, run on 6 real sources

Pipeline chosen

Source-grounded study notes, taken directly from a real line in my FL-01 workflow audit: “Summarizing ML research papers and library documentation” (classified: Delegate to AI with review). Applied here to AI/LLM security research — my actual professional domain — so the output is genuinely useful, not a throwaway exercise.

Tool: a Claude Project with structured, staged instructions — not NotebookLM or n8n. Chosen for the same reason as the Three Roads stack decision: free, zero new account, and matched to a beginner's real skill level. A single Project holding four sequential prompt templates is a genuine no-code workflow — chaining steps without writing code.

Pipeline design — 4 steps, defined handoffs

Step	Name	Input	Output → feeds into
1	Gather	Raw source (URL or pasted text)	Structured extraction: claims, evidence, methodology, direct citations → Step 2
2	Synthesize	Step 1's structured extraction	Themed organization + “relevance to my work” tags → Step 3
3	Draft	Step 2's themed synthesis	Clean, formatted study-note document → Step 4
4	Review	Step 3's draft + original source	Hallucination/citation check, flags for human verification → final note

Exact prompts used (the Project's staged instructions)

Step 1 — Gather

“Extract from this source, in your own words (no long quotes): (a) the core claim(s), (b) the evidence or methodology behind each claim, (c) any concrete numbers/findings, (d) the source's own citations for major claims. Flag anything you cannot verify from the text itself.”

Step 2 — Synthesize

“Take this extraction and organize it into 3–5 themes. For each theme, add one line: ‘relevant to my work because —’ tying it to building/testing secure LLM applications. Note any theme that contradicts or complicates what a previous source in this set said.”

Step 3 — Draft

“Format the themed synthesis into a clean study note: a one-line summary at the top, then each theme as a heading with 2–4 bullet points. Keep it skimmable — this should be readable in under 2 minutes.”

Step 4 — Review

"Compare the draft note against the original source. Flag: (a) any claim in the note not actually supported by the source, (b) any number that doesn't match, (c) anything stated with more confidence than the source itself used. List what a human must verify before trusting this note."

Verification run — brand new input, run live by hand

Run by hand in a Claude Project, real time: 3 minutes — a source not among the other five in this document. Source: Wu, Zhang, Jha, McDaniel & Xiao, "A New Era in LLM Security" (arXiv:2402.18649). This is the end-to-end proof that the pipeline runs on a new input, not just the ones already built into this walkthrough.

Step 1 (Gather): extracted the paper's core claim — LLM apps must be secured as whole systems (model + frontend + sandbox + plugins), not the model alone — plus its methodology (manual red-teaming against live GPT-4), concrete findings (a 4-step trick bypassed OpenAI's Safe URL Check; a chat-history exfiltration attack succeeded across all 8 tested web plugins, 100% success), and an explicit self-flagged limit: single-vendor, self-reported, no third-party replication.

Step 2 (Synthesize): organized into 5 themes — treat the app as a system, a constraint existing $\neq$ a constraint holding, indirect injection via tools as the main attack surface, session isolation as a separate infra risk, and the paper's own proposed mitigations — each tagged with why it matters for my own case study.

Step 3 (Draft): formatted into a clean study note with a one-line summary and 5 headed sections. Read well — and turned out to overclaim in two places, which Step 4 then caught.

Step 4 (Review) — the important part: caught two real problems in its own Step 3 draft, not hypothetical ones:

- The draft called the paper's proposed mitigations "concrete, implementable patterns... worth adopting now." The source explicitly says designing a defense "falls outside the scope of this study" — they're untested discussion points, not validated fixes. The draft overstated them.
- The draft's one-line summary said GPT-4's "safety constraints exist but are reliably bypassable" as a general claim. The paper only tested and broke three specific constraints plus one missing one — it never claims constraints are bypassable in general.
- The draft also dropped the source's own hedging language ("black-box," "we found," "based on our observation") and read as a settled audit instead of inferred, self-reported findings.

Step 4 also produced a 6-item human-verification list (current patch status, no independent replication, inferred vs. confirmed mechanism, scope of "bypassed," unvalidated mitigations, definition fidelity) — exactly the kind of check a human still has to do before trusting the note, which the pipeline surfaces rather than hides.

Why this matters for the deliverable: this run is the one place in this document where the pipeline's own review step visibly did its job — it didn't just pass the draft through, it caught the draft overselling its source. That's the workflow working as designed, on a source none of the earlier five runs used.

Five earlier runs (pipeline design and prompt testing)

All five sources are real, fetched live — not fabricated. Each run below is condensed to the Step 3 output plus the Step 4 review flag, to keep this readable; full Step 1/2 outputs were produced for each but aren't reproduced in full here.

Run 1 — OWASP LLM01:2025 Prompt Injection

genai.owasp.org/llmrisk/llm01-prompt-injection

One-line summary: OWASP's official taxonomy splits prompt injection into direct and indirect, and treats it as structurally unsolved rather than patchable.

Threat model: direct injection (user-typed) vs. indirect (via retrieved web/file content) vs. nine named attack scenarios, from payload splitting to multimodal hiding.

Defense posture: OWASP's own seven mitigations (constrain behavior, validate output format, filter input/output, least privilege, human approval, segregate untrusted content, adversarial testing) are framed as risk-reducers, not a fix — matches this project's own 'nothing to leak' architecture over relying on model behavior alone.

Relevance to my work: this is the canonical vocabulary (direct/indirect) my own case study's threat model already uses — confirms the categories, doesn't add a new one.

Step 4 review flag: Step 4 flagged nothing false — OWASP's page is itself the primary standard, so there's little to fact-check it against. Human check still needed: confirm this is still the current LLM01:2025 revision, since OWASP updates this list.

Run 2 — Sysdig, “The Comprehensive Guide to Prompt Injection Attacks in 2026”

sysdig.com/learn-cloud-native/prompt-injection

One-line summary: real incidents (EchoLeak, Cursor RCE, GitHub MCP) show the risk moved from chatbot embarrassment to agent-executed actions, and no single defense layer holds on its own.

The 'lethal trifecta': an agent is exploitable when it has private-data access, exposure to untrusted content, and outbound communication — all three together, named by researcher Simon Willison.

Defense-in-depth: architectural prevention (blast-radius limiting), runtime detection (catches consequences, not intent), and governance (ownership) — each layer catches something the others miss, none is sufficient alone.

Relevance to my work: the scholarship assistant's isolation design is exactly 'architectural prevention' — this source gives the vocabulary to explain *why* that layer was chosen first.

Step 4 review flag: Step 4 flagged one thing to verify: the claim that adaptive attacks bypass '90% of published defenses' cites a footnote number in the original article that isn't independently checkable from this page alone — needs the underlying paper before quoting that figure anywhere.

Run 3 — Liu et al., “Prompt Injection attack against LLM-integrated Applications” (arXiv:2306.05499)

arxiv.org/abs/2306.05499

One-line summary: an early academic paper (2023, revised 2025) that built a working black-box attack technique (HouYi) and found it worked against most real commercial apps it was tested on.

Method: HouYi splits an injection into three parts — a seamless pre-prompt, an injection that breaks context, and the actual malicious payload.

Result: tested on 36 real LLM-integrated apps, 31 were found vulnerable; 10 vendors (including Notion) confirmed the findings.

Relevance to my work: this is a primary source for 'this isn't theoretical' — useful to cite directly when a technical reader (like an Eng Lead) wants a real academic reference, not just blog posts.

Step 4 review flag: Step 4 flagged that this is an abstract-page fetch, not the full paper — the 31/36 figure and vendor list come from the abstract itself, which is fine to cite, but methodology details (how HouYi's three parts are actually constructed) would need the full PDF before I could explain the technique step by step.

Run 4 — Gulyamov et al., comprehensive review (Preprints.org / later published in MDPI Information)

preprints.org/manuscript/202511.0088

One-line summary: a large synthesis (120+ sources) documenting real 2024–2025 incidents (GitHub Copilot RCE, CamoLeak, Slack AI exfiltration) and arguing prompt injection is an architectural property of LLMs, not a bug to patch.

Headline finding: five crafted documents among millions can manipulate RAG responses 90% of the time — attributed to a specific PoisonedRAG-style study cited in the review, not measured by this review's own authors.

Named real incidents: CVE-2025-53773 (Copilot RCE), CamoLeak (CVSS 9.6 private repo exfiltration), and a

SCADA case where hidden PDF instructions caused physical industrial equipment damage.

Relevance to my work: the SCADA case is the strongest evidence I've seen that 'nothing to leak' architecture matters beyond text chatbots — physical consequences raise the stakes of the same isolation principle.

Step 4 review flag: Step 4's most important flag: the '90% RAG poisoning' number is repeated in this review but originates from a different, external study — the note must cite the original PoisonedRAG-style research, not this review, when quoting that figure. Also: this is an unreviewed preprint (later published in MDPI Information, 2026, 17(1):54) — worth citing the peer-reviewed version once confirmed identical.

Run 5 — Lan, Kaul & Jones (eBay), “Prompt Injection Detection in LLM Integrated Applications”
journal.hep.com.cn/ijndi/EN/10.53941/ijndi.2025.100013

One-line summary: an industry paper (eBay's security team) proposing a practical detection approach — a banned-term list combined with embedding similarity search, plus a fine-tuned BERT classifier — to catch injection attempts in real time.

Method (from the abstract only): combines a predefined banned-term list with embedding-based similarity search, and trains a BERT model as a classifier.

Positioning: explicitly framed as balancing security against usability, and acknowledges attack vectors keep evolving — doesn't claim the detector is complete.

Relevance to my work: this is the 'runtime detection' layer from Run 2's three-layer framework, in concrete, applied form — useful as a named example of what that layer looks like in practice.

*Step 4 review flag: Step 4 flagged the biggest gap of the five runs: I could only access the abstract and reference list, not the full paper (behind a login wall). No precision/recall numbers for the BERT classifier are available from what I fetched — anything about how *well* it detects injections would be guessing. Needs the full PDF before this note can say more than 'here is the method they used.'*

Time-saved estimate

The verification run gives a real, measured number: 3 minutes, hands-on, for one source through all 4 steps. The five earlier runs were done conversationally rather than timed with a stopwatch, so those are estimates — flagged as such below rather than presented as precise.

	Manual (est.)	Pipeline
Per source	30–45 min (read the paper, take notes, organize by hand)	3 min measured (verification run); ~5–8 min est. for the other 5
6 sources	~3–4.5 hours	~25–35 min, plus setup
One-time setup cost	n/a	~20–30 min to write and test the 4 prompt templates

Net: even counting setup cost, six sources via the pipeline is roughly 45–60 min versus 3–4.5 hours manually — and the one measured run (3 minutes) came in faster than the conversational estimate for the other five, suggesting those estimates are, if anything, conservative.

Where it breaks, and what a human must still check

- **The pipeline overclaims if you don't run Step 4 (verification run):** Step 3's draft on a brand-new source called untested proposals “concrete... worth adopting now” and turned a narrow finding into a general one. Step 4 caught both — but skipping Step 4 to save time would ship an overconfident note.

- **Paywalled/login-walled sources (Run 5):** the pipeline can only work with what it can actually fetch. An abstract-only fetch produces an abstract-only note — it will not silently invent the missing methodology, but a careless read of the output could mistake 'summarized the abstract' for 'summarized the paper.'
- **Numbers that travel between sources (Run 4):** the '90% RAG poisoning' figure appears in a review citing someone else's study. Step 4 caught this, but only because it was told to look for it — a differently-worded Step 4 prompt might have missed that the number wasn't the review's own finding.
- **Preprints vs. peer review (Run 4):** the pipeline doesn't know a source is an unreviewed preprint unless the source page says so explicitly. A human still has to check publication status before treating a claim as settled.
- **Step 4 can only check against what Step 1 extracted:** if Step 1 missed a nuance in the original source, Step 4 has nothing to compare it against and can't catch that specific gap — the review step verifies internal consistency, not omniscience.
- **Required human review, every run:** confirm the source is current (standards like OWASP's list get revised), confirm any cited statistic traces to its original study rather than a source repeating it, and confirm claims aren't stated more confidently in the note than they were in the source — exactly the class of error the verification run's Step 4 actually caught.